JavaScript is disabled on your browser.



compact1, compact2, compact3

java.lang.reflect

# Class Executable

java.lang.Object
    java.lang.reflect.AccessibleObject
        java.lang.reflect.Executable

**All Implemented Interfaces:**

AnnotatedElement, GenericDeclaration, Member

**Direct Known Subclasses:**

Constructor, Method

---

```
public abstract class Executable
extends AccessibleObject
implements Member, GenericDeclaration
```

A shared superclass for the common functionality of Method and Constructor.

**Since:**

1.8

## *Field Summary*

### Fields inherited from interface java.lang.reflect.Member

DECLARED, PUBLIC

## *Method Summary*

Concrete Methods

| Modifier and Type | Method | Descri |
| --- | --- | --- |
| AnnotatedType[] | getAnnotatedExceptionTypes() | Return Annota object repres types declar of the constr repres Execu |
| AnnotatedType[] | getAnnotatedParameterTypes() | Return Annota object repres types formal types constr repres Execu |
| AnnotatedType | getAnnotatedReceiverType() | Return Annota object repres a type receiv metho repres Execu |
| abstract AnnotatedType | getAnnotatedReturnType() | Return Annota object repres a type return metho repres Execu |
| <T extends Annotation> T | getAnnotation(Class<T> annotationClass) | Return eleme for the if such is *pres* |

| | | |
|---|---|---|
| <T extends **Annotation**> T[] | **getAnnotationsByType**(**Class**<T> annotationClass) | Return that a with t |
| **Annotation**[] | **getDeclaredAnnotations**() | Return that a *preser* eleme |
| abstract **Class**<?> | **getDeclaringClass**() | Return object the cla that d execut repres object |
| abstract **Class**<?>[] | **getExceptionTypes**() | Return `Class` repres of exc declar throw underl execut repres object |
| **Type**[] | **getGenericExceptionTypes**() | Return Type c repres except to be t execut |
| **Type**[] | **getGenericParameterTypes**() | Return Type c repres param declar the ex repres object |
| abstract int | **getModifiers**() | Return langua for the repres object |

| | | |
|---|---|---|
| abstract **String** | **getName**() | Return the ex repres object |
| abstract **Annotation**[][] | **getParameterAnnotations**() | Return arrays Annot repres annota formal in dec of the repres object |
| int | **getParameterCount**() | Return of form (wheth declar implic or neit execut repres object |
| **Parameter**[] | **getParameters**() | Return Param that re param underl execut repres object |
| abstract **Class**<?>[] | **getParameterTypes**() | Return Class repres param declar the ex repres object |
| abstract **TypeVariable**<?>[] | **getTypeParameters**() | Return TypeV object repres variab |

| | | |
|---|---|---|
| | | by the declar repres Gener object order. |
| boolean | **isSynthetic**() | Return execut synthe return otherv |
| boolean | **isVarArgs**() | Return execut declar variab argum false |
| abstract **String** | **toGenericString**() | Return descri Execu includ param |

### Methods inherited from class java.lang.reflect.AccessibleObject

getAnnotations, getDeclaredAnnotation, getDeclaredAnnotationsByType, isAccessible, isAnnotationPresent, setAccessible, setAccessible

### Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

### Methods inherited from interface java.lang.reflect.AnnotatedElement

getAnnotations, getDeclaredAnnotation, getDeclaredAnnotationsByType, isAnnotationPresent

## *Method Detail*

### getDeclaringClass

```
public abstract Class<?> getDeclaringClass()
```

Returns the `Class` object representing the class or interface that declares the executable represented by this object.

**Specified by:**

`getDeclaringClass` in interface `Member`

**Returns:**

an object representing the declaring class of the underlying member

### getName

```
public abstract String getName()
```

Returns the name of the executable represented by this object.

**Specified by:**

`getName` in interface `Member`

**Returns:**

the simple name of the underlying member

### getModifiers

```
public abstract int getModifiers()
```

Returns the Java language modifiers for the executable represented by this object.

**Specified by:**

`getModifiers` in interface `Member`

**Returns:**

the Java language modifiers for the underlying member

**See Also:**

`Modifier`

### getTypeParameters

```
public abstract TypeVariable<?>[] getTypeParameters()
```

Returns an array of `TypeVariable` objects that represent the type variables declared by the generic declaration represented by this `GenericDeclaration` object, in declaration order. Returns an array of length 0 if the underlying generic declaration declares no type variables.

**Specified by:**

`getTypeParameters` in interface `GenericDeclaration`

**Returns:**

an array of TypeVariable objects that represent the type variables declared by this generic declaration

**Throws:**

`GenericSignatureFormatError` - if the generic signature of this generic declaration does not conform to the format specified in *The Java™ Virtual Machine Specification*

### getParameterTypes

```
public abstract Class<?>[] getParameterTypes()
```

Returns an array of `Class` objects that represent the formal parameter types, in declaration order, of the executable represented by this object. Returns an array of length 0 if the underlying executable takes no parameters.

**Returns:**

the parameter types for the executable this object represents

### getParameterCount

```
public int getParameterCount()
```

Returns the number of formal parameters (whether explicitly declared or implicitly declared or neither) for the executable represented by this object.

**Returns:**

The number of formal parameters for the executable this object represents

### getGenericParameterTypes

```
public Type[] getGenericParameterTypes()
```

Returns an array of `Type` objects that represent the formal parameter types, in declaration order, of the executable represented by this object. Returns an array of length 0 if the underlying executable takes no parameters.

If a formal parameter type is a parameterized type, the Type object returned for it must accurately reflect the actual type parameters used in the source code.

If a formal parameter type is a type variable or a parameterized type, it is created. Otherwise, it is resolved.

**Returns:**

an array of Types that represent the formal parameter types of the underlying executable, in declaration order

**Throws:**

GenericSignatureFormatError - if the generic method signature does not conform to the format specified in *The Java™ Virtual Machine Specification*

TypeNotPresentException - if any of the parameter types of the underlying executable refers to a non-existent type declaration

MalformedParameterizedTypeException - if any of the underlying executable's parameter types refer to a parameterized type that cannot be instantiated for any reason

### getParameters

```
public Parameter[] getParameters()
```

Returns an array of Parameter objects that represent all the parameters to the underlying executable represented by this object. Returns an array of length 0 if the executable has no parameters.

The parameters of the underlying executable do not necessarily have unique names, or names that are legal identifiers in the Java programming language (JLS 3.8).

**Returns:**

an array of Parameter objects representing all the parameters to the executable this object represents.

**Throws:**

MalformedParametersException - if the class file contains a MethodParameters attribute that is improperly formatted.

### getExceptionTypes

```
public abstract Class<?>[] getExceptionTypes()
```

Returns an array of Class objects that represent the types of exceptions declared to be thrown by the underlying executable represented by this

object. Returns an array of length 0 if the executable declares no exceptions in its `throws` clause.

**Returns:**

the exception types declared as being thrown by the executable this object represents

### getGenericExceptionTypes

```
public Type[] getGenericExceptionTypes()
```

Returns an array of `Type` objects that represent the exceptions declared to be thrown by this executable object. Returns an array of length 0 if the underlying executable declares no exceptions in its `throws` clause.

If an exception type is a type variable or a parameterized type, it is created. Otherwise, it is resolved.

**Returns:**

an array of Types that represent the exception types thrown by the underlying executable

**Throws:**

`GenericSignatureFormatError` - if the generic method signature does not conform to the format specified in *The Java™ Virtual Machine Specification*

`TypeNotPresentException` - if the underlying executable's throws clause refers to a non-existent type declaration

`MalformedParameterizedTypeException` - if the underlying executable's throws clause refers to a parameterized type that cannot be instantiated for any reason

### toGenericString

```
public abstract String toGenericString()
```

Returns a string describing this `Executable`, including any type parameters.

**Returns:**

a string describing this Executable, including any type parameters

### isVarArgs

```
public boolean isVarArgs()
```

Returns `true` if this executable was declared to take a variable number of arguments; returns `false` otherwise.

**Returns:**

`true if an only if this executable was declared to take a variable number of arguments.`

### isSynthetic

```
public boolean isSynthetic()
```

Returns `true` if this executable is a synthetic construct; returns `false` otherwise.

**Specified by:**

`isSynthetic` in interface `Member`

**Returns:**

`true if and only if this executable is a synthetic construct as defined by *The Java™ Language Specification*.`

**See *The Java™ Language Specification*:**

`13.1 The Form of a Binary`

### getParameterAnnotations

```
public abstract Annotation[][] getParameterAnnotations()
```

Returns an array of arrays of `Annotations` that represent the annotations on the formal parameters, in declaration order, of the `Executable` represented by this object. Synthetic and mandated parameters (see explanation below), such as the outer "this" parameter to an inner class constructor will be represented in the returned array. If the executable has no parameters (meaning no formal, no synthetic, and no mandated parameters), a zero-length array will be returned. If the `Executable` has one or more parameters, a nested array of length zero is returned for each parameter with no annotations. The annotation objects contained in the returned arrays are serializable. The caller of this method is free to modify the returned arrays; it will have no effect on the arrays returned to other callers. A compiler may add extra parameters that are implicitly declared in source ("mandated"), as well as parameters that are neither implicitly nor explicitly declared in source ("synthetic") to the parameter list for a method. See `Parameter` for more information.

**Returns:**

`an array of arrays that represent the annotations on the formal and implicit parameters, in declaration order, of the executable represented by this object`

**See Also:**

Parameter, Parameter.getAnnotations()

### getAnnotation

```
public <T extends Annotation> T getAnnotation(Class<T> annotationClass)
```

Returns this element's annotation for the specified type if such an annotation is *present*, else null.

**Specified by:**

getAnnotation in interface AnnotatedElement

**Overrides:**

getAnnotation in class AccessibleObject

**Type Parameters:**

T - the type of the annotation to query for and return if present

**Parameters:**

annotationClass - the Class object corresponding to the annotation type

**Returns:**

this element's annotation for the specified annotation type if present on this element, else null

**Throws:**

NullPointerException - if the given annotation class is null

### getAnnotationsByType

```
public <T extends Annotation> T[] getAnnotationsByType(Class<T> annotationClass)
```

Returns annotations that are *associated* with this element. If there are no annotations *associated* with this element, the return value is an array of length 0. The difference between this method and AnnotatedElement.getAnnotation(Class) is that this method detects if its argument is a *repeatable annotation type* (JLS 9.6), and if so, attempts to find one or more annotations of that type by "looking through" a container annotation. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

**Specified by:**

getAnnotationsByType in interface AnnotatedElement

**Overrides:**

getAnnotationsByType in class AccessibleObject

**Type Parameters:**

T - the type of the annotation to query for and return if present

**Parameters:**

annotationClass - the Class object corresponding to the annotation type

**Returns:**

all this element's annotations for the specified annotation type if associated with this element, else an array of length zero

**Throws:**

NullPointerException - if the given annotation class is null

### getDeclaredAnnotations

public Annotation[] getDeclaredAnnotations()

Returns annotations that are *directly present* on this element. This method ignores inherited annotations. If there are no annotations *directly present* on this element, the return value is an array of length 0. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

**Specified by:**

getDeclaredAnnotations in interface AnnotatedElement

**Overrides:**

getDeclaredAnnotations in class AccessibleObject

**Returns:**

annotations directly present on this element

### getAnnotatedReturnType

public abstract AnnotatedType getAnnotatedReturnType()

Returns an AnnotatedType object that represents the use of a type to specify the return type of the method/constructor represented by this Executable. If this Executable object represents a constructor, the AnnotatedType object represents the type of the constructed object. If this Executable object represents a method, the AnnotatedType object represents the use of a type to specify the return type of the method.

**Returns:**

an object representing the return type of the method or constructor represented by this Executable

### getAnnotatedReceiverType

```
public AnnotatedType getAnnotatedReceiverType()
```

Returns an `AnnotatedType` object that represents the use of a type to specify the receiver type of the method/constructor represented by this Executable object. The receiver type of a method/constructor is available only if the method/constructor has a *receiver parameter* (JLS 8.4.1). If this `Executable` object represents a constructor or instance method that does not have a receiver parameter, or has a receiver parameter with no annotations on its type, then the return value is an `AnnotatedType` object representing an element with no annotations. If this `Executable` object represents a static method, then the return value is null.

**Returns:**

an object representing the receiver type of the method or constructor represented by this Executable

### getAnnotatedParameterTypes

```
public AnnotatedType[] getAnnotatedParameterTypes()
```

Returns an array of `AnnotatedType` objects that represent the use of types to specify formal parameter types of the method/constructor represented by this Executable. The order of the objects in the array corresponds to the order of the formal parameter types in the declaration of the method/constructor. Returns an array of length 0 if the method/constructor declares no parameters.

**Returns:**

an array of objects representing the types of the formal parameters of the method or constructor represented by this Executable

### getAnnotatedExceptionTypes

```
public AnnotatedType[] getAnnotatedExceptionTypes()
```

Returns an array of `AnnotatedType` objects that represent the use of types to specify the declared exceptions of the method/constructor represented by this Executable. The order of the objects in the array corresponds to the order of the exception types in the declaration of the method/constructor. Returns an array of length 0 if the method/constructor declares no exceptions.

**Returns:**

an array of objects representing the declared exceptions of the method or constructor represented by this Executable



Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.